

SK윌더스 루키즈 26기 (인포섹 아카데미)

지능형 축구 챗봇 개발: 정보분석, 뉴스요약, 승부예측

TEAM 2팀

AI Football Manager (아이에잇(I8))

오준서, 박정은, 백상준, 변창민,
원정임, 이동은, 임지혁, 정하은
[멘토] 유명진



Github



목 차

- 01 프로젝트 개요
- 02 프로젝트 팀 구성 및 역할
- 03 프로젝트 수행 절차 및 방법
- 04 프로젝트 수행 경과
- 05 자체 평가 의견

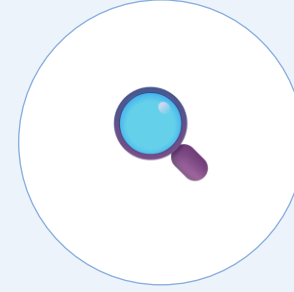
1 프로젝트 주제 및 선정 배경

**기술 트렌드**

GPT-4와 Function
Calling 기술의 발전

**도메인 특성**

축구는 데이터가 풍부하고
실시간 정보가 중요한
도메인

**문제 인식**

기존 챗봇은 단순 응답만
가능 → 분석/해설 부족

→ 따라서, GPT-4 Function Calling과 멀티 에이전트 구조를 활용한 지능형 축구 인터페이스를 개발하기로 하였다.

01

K-Digital Training

프로젝트 개요

1

프로젝트 기획의도

1

누구나 쉽게,
직관적인 축구 정보 접근

- 복잡한 명령 없이 **자연어 입력**만으로 정보 탐색
- 일반 팬부터 기자, 분석가까지 모두를 위한 설계
- 자연스러운 **대화형 인터페이스 UX**



2

AI가 전 과정을 자동 처리

- 사용자의 질문 → 자동 분류
- 데이터 수집 → 분석 → 요약, 해설까지 **AI가 자동 수행**
- 수작업 없이 질문만 하면 결과 제공



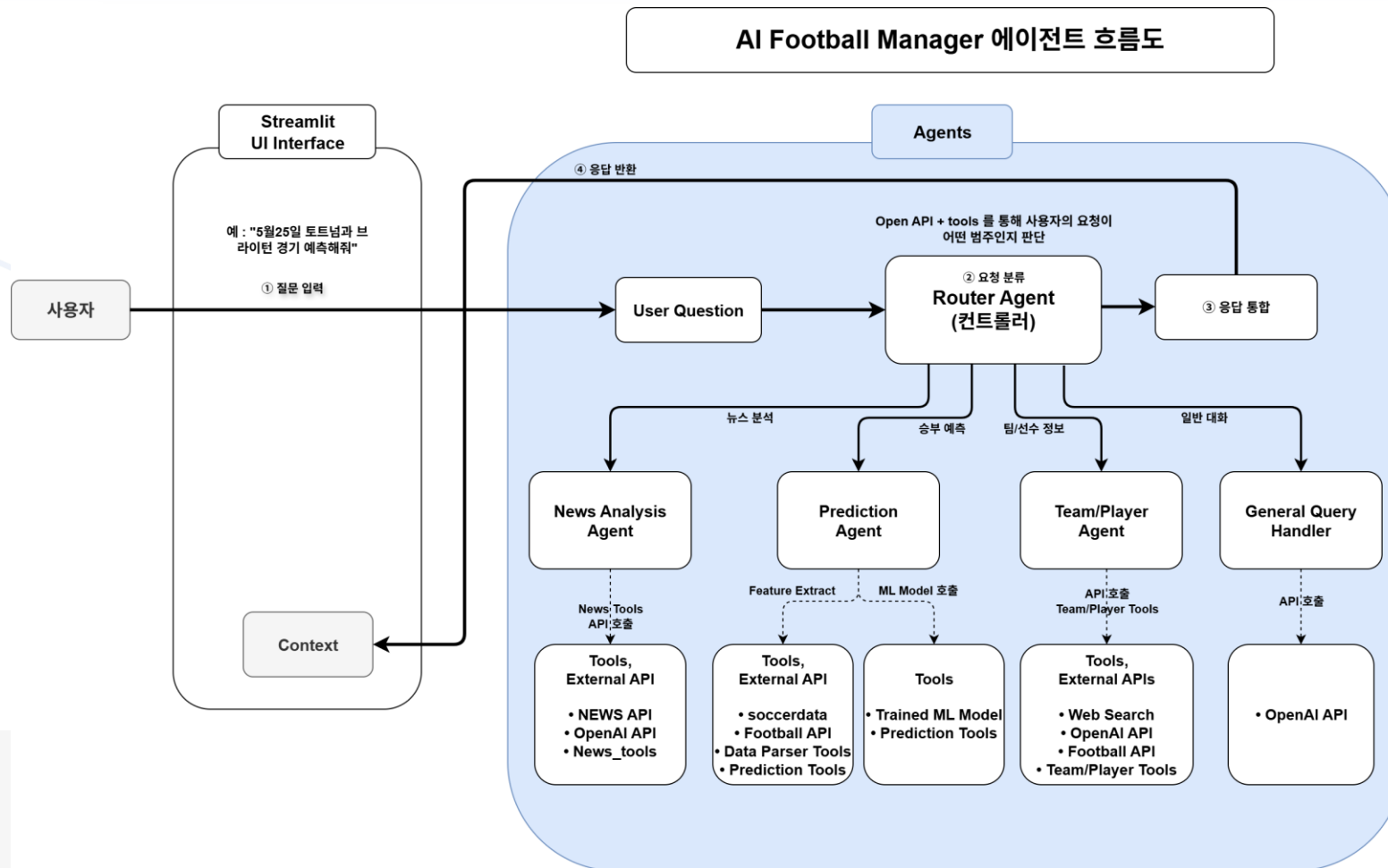
3

하나의 채팅창에서
다양한 기능 연결

- 뉴스 요약, 경기 예측, 선수 정보 조회를 하나의 채팅 인터페이스에서
- 정보 흐름이 끊기지 않고 자연스럽게 이어짐
- **통합형 UX** → 몰입도 높은 사용자 경험



➔ 단순 정보 조회를 넘어 AI 해석 + 해설이 결합된 지능형 시스템을 목표로 하였다.



router_agent

- 역할 : 사용자 질문 분석 및 기능 분기
- 처리 흐름 : 질문 + 대화 이력
→ GPT 4로 Function 선택
→ 해당 기능 호출

news_analysis_agent

- 역할 : 축구 뉴스 수집 및 분석
- 처리 흐름 : 키워드 추출
→ 뉴스 수집
→ 요약 + 감정 분석 + AI 코멘트 생성

prediction_agent

- 역할 : 경기 결과 예측 및 해결
- 처리 흐름 : 팀명/날짜 파싱
→ 통계 수집 → ML 모델 예측
→ AI 해설 생성

team_player_agent

- 역할 : 선수/팀 통계 제공
- 처리 흐름 : 대상 추출
→ API 조회
→ 통계 요약 및 설명 응답 생성


Agents

항목	이름	설명
모델	GPT-4	GPT-4.1은 질문 분류 및 라우팅에, GPT-4o는 각 기능별 응답 생성에 사용
프레임워크	Streamlit	사용자 입력 및 응답 출력을 위한 웹 인터페이스 구현
라이브러리	scikit-learn	경기 예측을 위한 머신러닝 모델 학습 및 예측
	pandas	데이터 처리 및 구조화
	tiktoken	GPT와의 통신을 위한 토큰 수 계산
	soccerdata	경기 통계 수집용 라이브러리 (Elo, xG 등)
외부 API	API-Football	선수 및 팀의 시즌별 실시간 통계 정보 수집
	News API	최신 축구 기사 검색 및 수집
하드웨어	로컬 CPU 환경	GPU 없이도 실행 가능, 일반 노트북에서도 충분히 구동 가능

01

K-Digital Training

프로젝트 개요

4

프로젝트 구조

AI-Football-Manager/

├── ai_football_manager/

│ ├── agents/

│ │ ├── router_agent.py

│ │ ├── news_analysis_agent.py

│ │ ├── prediction_agent.py

│ │ └── team_player_agent.py

│ ├── tools/

│ │ ├── sports_data_api.py

│ │ └── ...

│ ├── utils/

│ ├── models/

│ ├── app.py

│ └── ...

└── ...

메인 애플리케이션 패키지

AI 에이전트 모듈 : 기능별 전문 에이전트

질문 분류 및 라우팅 (다중 도구 호출 지원) : GPT-4 사용, 질문의 의미 해석 → 기능 분기

뉴스 분석 전문 에이전트

경기 예측 전문 에이전트

팀/선수 정보 전문 에이전트

데이터 처리 도구 : GPT가 호출 가능한 Function 정의 (JSON 스키마)

스포츠 데이터 API 인터페이스

유틸리티 모듈 : 프롬프트 관리, 토큰 처리, 외부 데이터 수집 등 보조 기능

머신러닝 모델 저장소

Streamlit 웹 애플리케이션 : 사용자 입력 수집, router 호출, 응답 출력



일반 축구 팬

- “손흥민 요즘 어때?”, “맨유 분위기 어때?”
→ 자연어로 간단히 질의
- 뉴스 + 통계 + 코멘트를 한 번에 확인 가능
- 별도 앱/전문지식 불필요



스포츠 분석가

- 지능형 라우팅 + 다중 도구 호출 기능 활용
→ 복합 질문에 대한 전문 데이터
통합 분석 자동화



축구 기자 및 콘텐츠 제작자

- 기사 작성 전 AI가 제공하는 뉴스 요약,
감정 분석, 한 줄 코멘트 참고
- 경기 예측 통계 자료도 활용 가능
- 작성 시간 절약 + 정보 정확도 향상



교육 및 연구

- 멀티 에이전트, NLP,
프롬프트 설계 등의 사례로 사용
- AI 활용 경험의 확대

01

K-Digital Training

프로젝트 개요

5

기대 효과

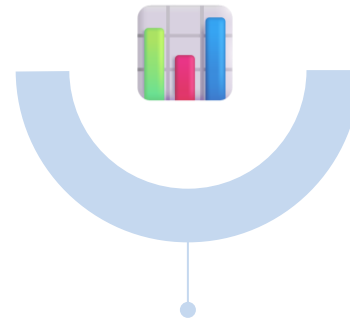
정보 탐색 흐름 자연스러움

뉴스 → 선수 → 경기로 이어지는
자연스러운 인터랙션 환경



사용자 접근성 향상

질문만으로 뉴스/통계/예측까지
탐색 가능 → UX 극대화

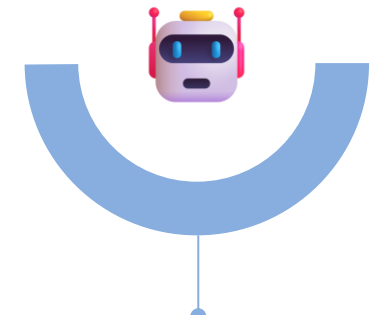
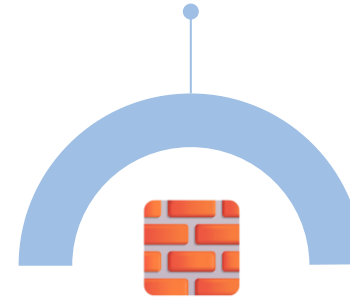


데이터 기반 의사결정 지원

분석 자료 기반의
마케팅/전략 수립 가능

서비스 확장성 확보

에이전트 추가만으로
손쉬운 기능 및 도메인 확장



GPT 기반 AI 활용 경험 확대

GPT를 단순 Q&A가 아닌
기능형 도구로 활용

프로젝트 팀 구성 및 역할

▶ 프로젝트 구현 목표

1. 머신러닝을 활용한 승부예측 - 정확도 90% 이상

- 과거 경기 데이터를 활용, Elo, xG, 최근 전적, 순위, 홈/원정 여부 등 다양한 특성을 수집 및 가공
- 외부 API (api-football) 를 통해 실시간 경기 정보 수집 (승부 예측에 사용 → 예측을 위한 전처리 필요)
- 모델링 전략 : 다양한 예측 모델을 실험하고, 다양한 피처와 하이퍼파라미터 튜닝을 통해 성능을 최적화

2. 웹 페이지 크롤링을 통한 기사 요약 (web search 사용, 의미 있는 요약)

- 기사 내 핵심 키워드를 추출, 핵심 문장 추출을 통해 요약문 생성 → 이를 통해 질의응답

3. 통계 제공 및 데이터 시각화

- 팀별 리그 순위, 득실차, 홈/원정 승률, Elo rating, xG(기대득점), 예상 승리 팀, 팀별 선수 통계 (스탯, 부상유무)
- Matplotlib 를 활용해서 시각화 → streamlit을 통해 UI/UX 구성

4. 멀티에이전트 구조

- 전체 구조 : 사용자 → streamlit 인터페이스 → 컨트롤러
 - 컨트롤러가 요청의 종류(승부예측/기사요약/팀별분석/일반대화 등) 를 판별 → 요청에 따라 적절한 에이전트에 분배
- 기사요약 에이전트
 - 기사 크롤링 에이전트 : web search 및 BeautifulSoup 등으로 주요 뉴스 사이트에서 최신 기사 수집
 - 기사 요약 에이전트 : 기사 내 키워드, 핵심 문장 추출, OpenAI API를 통해 의미 있는 요약문 생성
- 승부예측 에이전트
 - 웹 검색 에이전트 : api-football, web search 등을 활용해 실시간 팀 정보 및 과거 데이터 수집
 - 전처리 에이전트 : Elo, xG, 최근 전적, 순위, 홈/원정 여부 등 다양한 피처 생성 및 정제 (실시간 예측에 사용하는 데이터로 비교하는 2개의 팀에 대한 데이터를 수집 및 전처리 필요)
 - 예측 전용 에이전트 : 학습된 모델을 통해 예측 결과 산출
- 시각화 에이전트
 - 통계 제공 및 시각화 : 팀/선수 별 데이터를 시각화Matplotlib로 그린 후 streamlit에 시각화 결과

02

K-Digital Training

프로젝트 팀 구성 및 역할

▶ 프로젝트 팀 구성 및 역할 - 1

훈련생	역할	담당 업무
오준서	팀장	프로젝트 리더, 데이터 수집 및 전처리, 승부 예측 에이전트 구현, 분류 에이전트 function calling 구현, 다중 도구 호출 구현(router), 서비스 통합, 에이전트 통합/테스트, 에이전트 흐름도 작성, 사용자 가이드 작성
이동은	팀원	데이터 수집 및 전처리, 승부 예측 모델 제작, 예측 데이터 수집 함수 제작, 모델 학습/사용법 문서 작성
정하은	팀원	데이터 탐색 및 전처리, 승부 예측 모델 제작, ppt 제작, 문서화 작업(주요 설계 구조 보고서, 사용자 가이드)
변창민	팀원	분류, 뉴스/분석, 팀/선수 에이전트 구현 관련 도구, 에이전트 통합, UI 구현, 문서화 작업(툴/함수 정의서, 사용자 가이드)

02

K-Digital Training

프로젝트 팀 구성 및 역할

▶ 프로젝트 팀 구성 및 역할 - 2

훈련생	역할	담당 업무
원정임	팀원	데이터 정제 및 정규화, 서비스 테스트
임지혁	팀원	외부 데이터 수집
백상준	팀원	뉴스/분석 도구 함수 구현, 크롤링 데이터 분리 프롬프트 제작, 문서화 작업
박정은	팀원	데이터 탐색 및 전처리, 승부 예측 모델 제작

03

K-Digital Training

프로젝트 수행 절차 및 방법

▶ 프로젝트의 사전 기획과 프로젝트 수행 및 완료 과정 - 1

구분	기간	활동	비고
사전 기획	7/1(화) ~ 7/1(화)	🚩 협업 레포지토리 생성 🚩 기획안 작성 🚩 필요 데이터 정의 및 수집 🚩 역할 분담	브레인스토밍을 통한 주제 선정
데이터 수집	7/1(화) ~ 7/2(수)	🚩 승부 예측 데이터 수집 🚩 데이터 수집 API 탐색 🚩 팀/선수 관련 데이터 수집	핵심 feature 및 데이터 수집 API 탐색에 집중
데이터 전처리	7/1(화) ~ 7/2(수)	🚩 수집 데이터 통합 🚩 API 활용 구현 🚩 카테고리 데이터 Encoding	
모델 학습	7/2(수) ~ 7/4(금)	🚩 RandomForest 학습 🚩 LightGBM 학습 🚩 XGBoost 학습 🚩 ExtraTreesClassifier 학습	다양한 모델을 학습한 후 정확도 비교

03

K-Digital Training

프로젝트 수행 절차 및 방법

▶ 프로젝트의 사전 기획과 프로젝트 수행 및 완료 과정 - 2

구분	기간	활동	비고
기사 크롤링 개발	7/1(화) ~ 7/4(금)	🚩 뉴스 크롤링 API 탐색 🚩 GPT 프롬프트 제작 🚩 뉴스 분석 및 요약 에이전트 개발	
함수 개발	7/2(수) ~ 7/6(일)	🚩 승부 예측 함수 개발 🚩 다중 function call 개발 🚩 과거 대화기반 맥락 유지 함수 개발	
서비스 통합	7/3(목) ~ 7/6(일)	🚩 디렉토리 구조화 진행 🚩 함수 기반 에이전트 개발 🚩 Streamlit과 에이전트 통합	챗봇 로딩 시간 단축
총 개발 시간	7/1(화) ~ 7/6(일) (총 8일)	🚩 승부 예측, 기사 요약, 선수 정보를 알려주는 챗봇 개발	

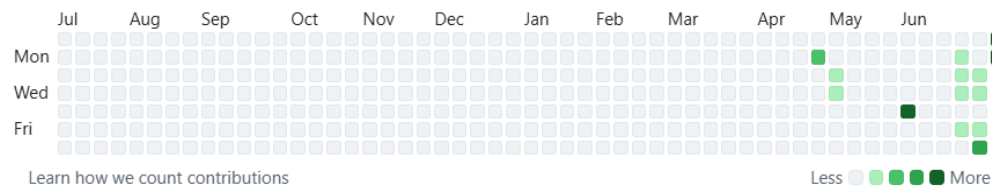
03

K-Digital Training

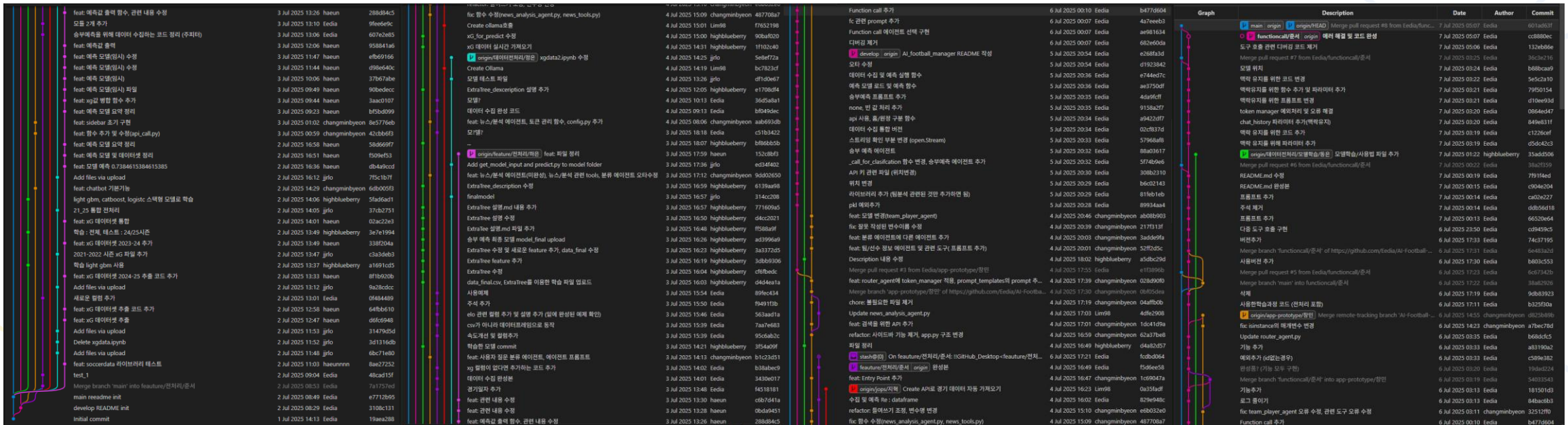
프로젝트 수행 절차 및 방법

▶ 구현 과정

7/1(화) ~ 7/7(월) 까지 프로젝트 구현을 진행



Git Graph



▶ 머신러닝 모델 - ExtraTreesClassifier

모델 채택 이유

- 트리 기반 모델은 데이터의 분포나 크기에 크게 의존 하지 않고 이진분류에 특화되어 있다.
- 적은 데이터셋으로도 랜덤성을 통해서 과적합을 방지한다.

Train vs Validation 성능 (mean $\pm$ std)acc : 0.952 $\pm$ 0.003 | 0.748 $\pm$ 0.029auc : 0.990 $\pm$ 0.001 | 0.817 $\pm$ 0.024logloss : -0.224 $\pm$ 0.007 | -0.540 $\pm$ 0.047

학습 방식

- 여러 개의 독립된 트리를 생성하고, 분기할 feature와 경계값을 무작위로 선택한다.
- 모든 트리의 결과값 모아 평균을 낸다.
- 한 분야에 대한 수백명의 전문가를 모아놓고, 이들에게 같은 문제를 고민하게 만들면 평균적으로 좋은 결과에 수렴하는 것과 같은 원리이다.

▶ 확률 교정 - CalibratedClassifierCV

확률 교정이란

샘플	모델 예측 확률 (긍정 : 1)	실제 결과
1	0.95	1
2	0.90	1
3	0.85	1
4	0.80	0
5	0.75	0
6	0.70	0

왼쪽의 경우에서 1~4에 해당하는 예측 확률은 87.5%이다.

하지만 실제로 예측이 맞은 경우는 4개 중 3개인 75%이다.
이러한 차이가 커질수록 모델의 신뢰도가 떨어진다.

그래서 CalibratedClassifierCV는 모델의 예측 결과를 바꾸지는 않지만
내부적으로 모델이 예측한 확률 값을 바꾸어 0.875를 실제 결과인 75%로
교정하는 것이다.

사용자에게 승부 예측 결과만이 아닌 확률도 보여주는 것이 중요하기 때문이다.

04

K-Digital Training

프로젝트 수행 경과

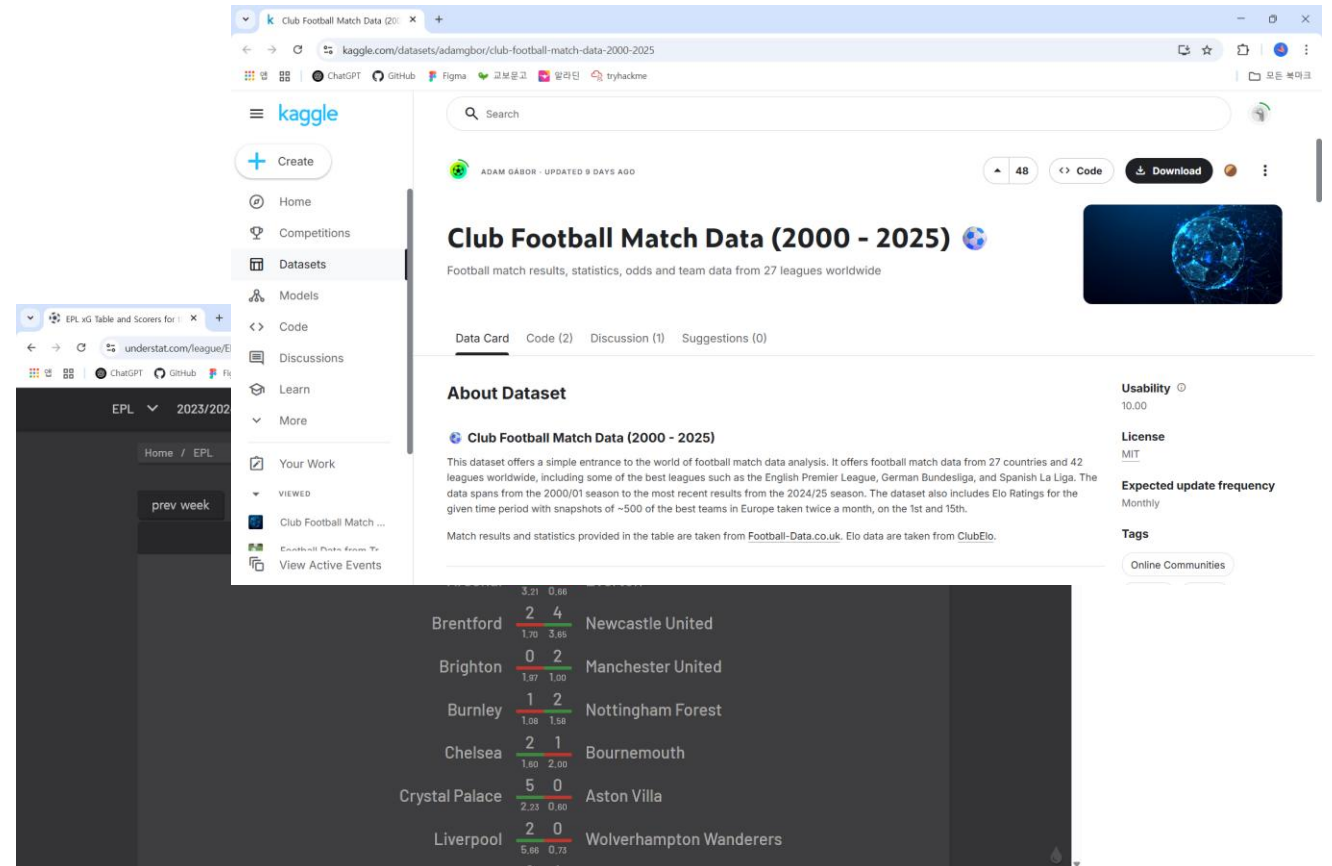
▶ 학습 데이터 소개

Matches.csv

- 2005년부터 2025년도까지의 축구 5대 리그에 대한 데이터
- 출처 : [kaggle](https://kaggle.com/datasets/adamgbor/club-football-match-data-2000-2025)

soccerdata 라이브러리

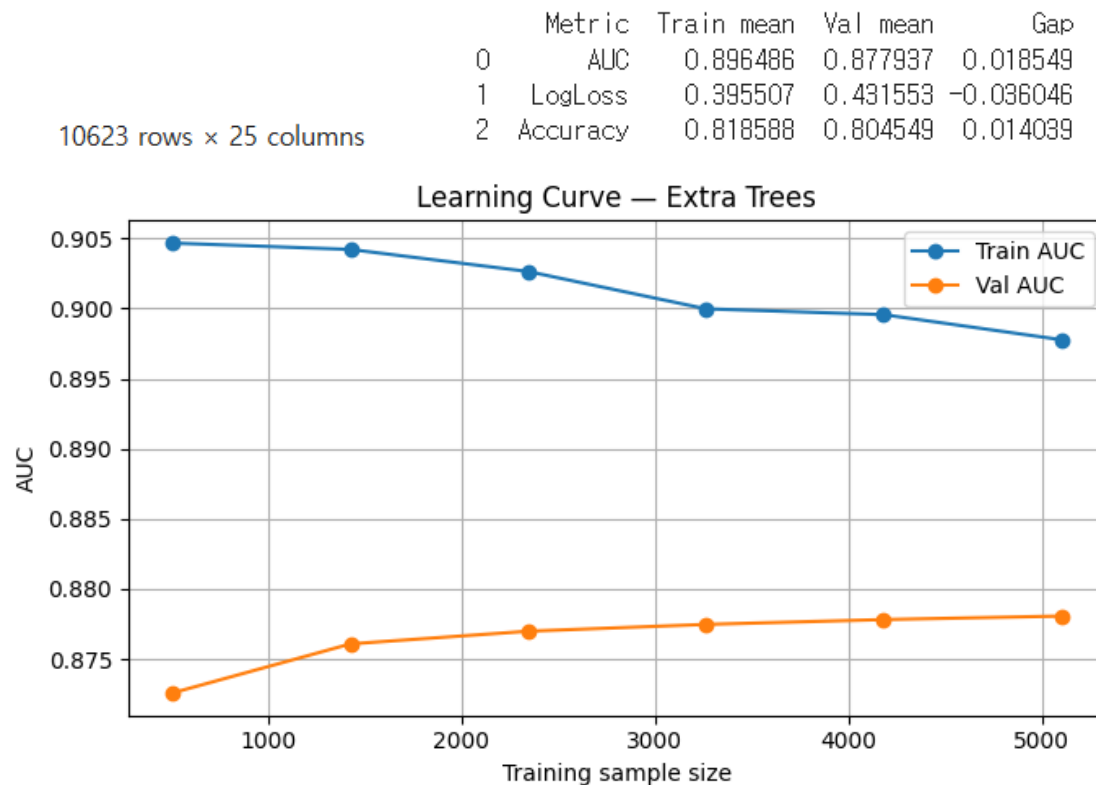
- 축구 5대 리그의 경기별 데이터 (골, 실점, 유효슈팅, xG)
- 출처 : <https://understat.com/league/EPL/2023>



▶ 실제 승부 예측을 위한 모델 학습 결과

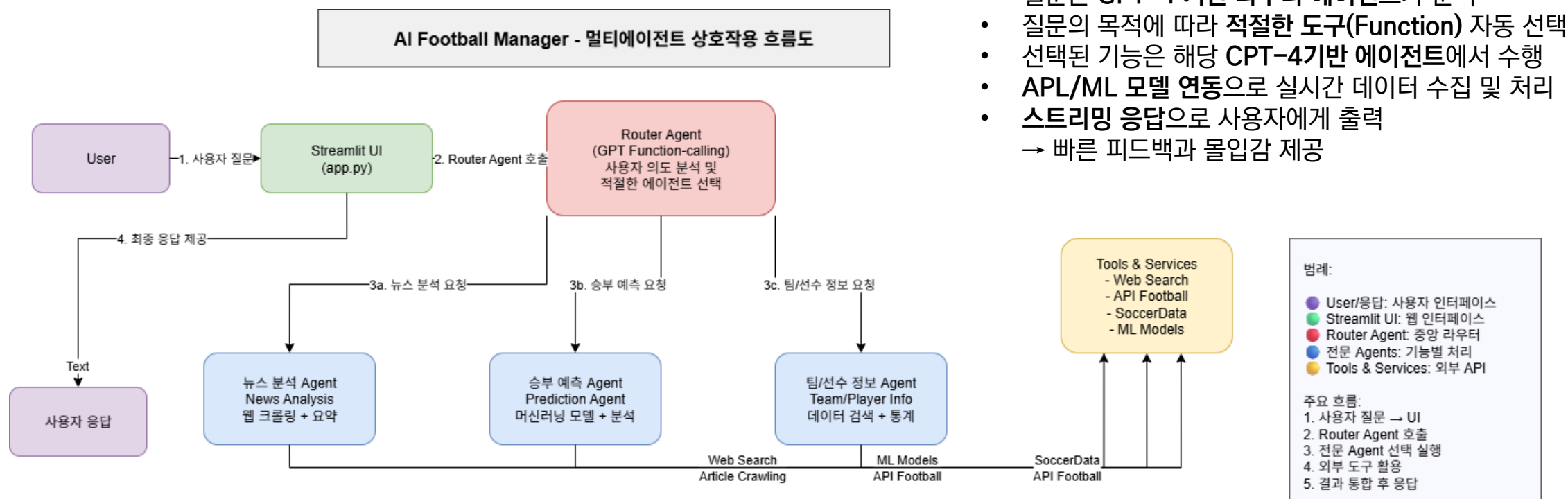
실제 예측 모델 학습에서는 EPL 이외에도 다른 빅리그들도 포함하여 데이터셋을 구성하였고 학습을 진행하였다.

구분	포함된 feature	목적
메타정보	MatchDate, HomeTeam, AwayTeam	경기 및 팀 정보
전력 지표	HomeElo, AwayElo, elo_diff	팀 기본 실력 지표(Elo)
최근 성적	Form3Home, Form5Home, Form3Away, Form5Away	최근 승점 수로 팀의 추세 파악
득·실점 추세	GF3Home, GA3Home, GF5Home, FA5Home, GF3Away, GA3Away, GF5Away, GA5Away	최근 경기에서 홈팀과 원정팀이 넣은 골 수, 실점 수 (F: 골 수, A: 실점 수)
xG 지표	H_xg, a_xg, xG_diff, xg_margin, xg_ratio, rolling_xg_home_5, rolling_xg_away_5	xG(Expected Goals), 기대 득점 수준 & 양팀 비교, margin(diff의 절댓값), ratio(홈과 원정팀의 xG비율)
경기 결과	FTResult	타겟 레이블(H, A), H - 홈팀 승리, A - 원정팀 승리



▶ 에이전트 구현 (필수 요구사항)

A. OpenAI API 및 에이전트 플랫폼 활용



- 사용자 질문은 Streamlit UI를 통해 입력됨
- 질문은 GPT-4 기반 라우터 에이전트가 분석
- 질문의 목적에 따라 적절한 도구(Function) 자동 선택
- 선택된 기능은 해당 CPT-4기반 에이전트에서 수행
- APL/ML 모델 연동으로 실시간 데이터 수집 및 처리
- 스트리밍 응답으로 사용자에게 출력
→ 빠른 피드백과 몰입감 제공

▶ 에이전트 구현 (필수 요구사항)

A. OpenAI API 및 에이전트 플랫폼 활용

- Chat completions API 사용
- 질문 의도를 파악하여 자동 분류를 하는데 GPT-4.1 모델을 사용하였고, 적절한 전문 에이전트로 연결을 위해서 OpenAI의 Function calling 기능을 활용하여 tools 리스트에 등록된 기능 중 GPT가 판단하여 적절한 함수 호출
- 복합 질문 시 여러 에이전트를 동시 실행하여 통합된 답변 제공

```
tools = [ get_team_player_info_tool, analyze_news_tool, predict_match_tool, handle_general_query_tool ]
```

- 각 역할을 맡은 하위 agent들이 정보를 찾지 못한 경우, 사용자 질문을 **파싱**하여 GPT에게 제공할 내용을 정제하는 과정에서 잘못되는 경우 등에서 GPT-4o 모델의 **호스팅 툴(Web search)**를 사용하여 검색하여 최종 응답을 생성

▶ 에이전트 구현 (필수 요구사항)

B. 커스텀 함수(툴) 구현

- 승부 예측을 하기위해 제작한 모델을 `models/final_model.pkl` 모델 파일로 저장
 - 모델 입력 : home/away team 최신화 된 features
 - 모델 출력 : home/away team 승리 확률
- 커스텀 툴(함수) : 커스텀 툴로 에이전트에 연결 → 사용자 입력에 따라 해당 에이전트로 각 기능을 수행
 - `get_team_player_info_tool` : 팀별/선수 별 상세 정보 제공 도구 (`team_player_agent` 호출)
 - `analyze_news_tool` : 기사 요약 및 분석 제공 도구 (`news_analysis_agent` 호출)
 - `predict_match_tool` : 승부예측 및 분석 제공 도구 (`prediction_agent` 호출)
 - `handle_general_query_tool` : 다른 범주에 속하지 않는 일반적인 축구 관련 질문 처리 제공 도구 (`prediction_agent` 호출)

▶ 에이전트 구현 (필수 요구사항)

C. 멀티에이전트 활용

- 메인 컨트롤러 에이전트 (rounter_agent) → 승부 예측 에이전트, 뉴스분석 에이전트, 팀정보 에이전트로 요청 분배
- 각 전문 에이전트가 독립적으로 작업 수행 후 결과를 메인 컨트롤러 에이전트로 반환
- 복합 요청 시 여러 에이전트가 순차적으로 협업 (예 : 뉴스 분석 → 팀 정보 조회 → 승부 예측 을 통합한 결과를 응답)

각 에이이전트의 전문화된 역할

- 뉴스분석 에이전트 : NEWS API로 기사 수집, 감정분석, 요약 전달
- 승부예측 에이전트 : 머신러닝 모델 추론 전달
- 팀정보 에이전트 : 통계 데이터 조회 및 시각화 전달

- 메인 컨트롤러 : 요청 분석, 에이전트 호출 순서 결정, 결과 통합 및 컨텍스트 관리

1. 단일 도구 호출 : 사용자가 “토트넘 전적 알려줘” → 컨트롤러가 팀 정보 에이전트만 호출 → 결과 반환

2. 다중 도구 호출 : 사용자가 “5월 25일 토트넘 경기 예측과 관련 최신 뉴스도 함께 알려줘” → 컨트롤러가 순차적으로 뉴스 에이전트 호출
→ 승부예측 에이전트 호출 (수집, 예측, 분석) → 모든 결과를 종합하여 통합된 분석 제시

▶ 에이전트 구현 (필수 요구사항)

D. 통합 대화 UX

- 사용자 입장에서는 자연어로 자유롭게 질문·지시 가능

“5월 18일 아스널 경기 예측 해줘”, “프리미어리그 최신 뉴스 요약해줘”,

“5월 25일 토트넘 경기 예측해줘, 상대팀인 브라이턴 전적 분석하고 종합적 분석 해줘”

- 복합 요구사항의 자동 오케스트레이션

- 최신 뉴스 + 예측 : “5월 18일 아스널 경기 예측하고 최근 소식 알려줘”, → 뉴스 에이전트 → 예측 모델 적용 → 뉴스 요약 및 분석과 예측 통합 제공
- 종합 분석 : “아스널 팀 정보, 최근 뉴스, 5월 18일 경기 예측 모두 알려줘” → 팀 정보 + 승부예측 + 뉴스분석을 순차 실행하여 통합 리포트 제공

- 지능형 맥락 유지 및 연속 대화

- 대화 히스토리 관리 : “아스널에 대해 알려줘” → “그럼 다음 경기는 언제야?” (이전 아스널 맥락 유지)

- Streamlit 기반 직관적 인터페이스

- 채팅형 대화 인터페이스로 자연스러운 질의응답

- 효율적인 토큰 관리 시스템

- 실시간 토큰 계산 : `get_token_count()` 함수로 GPT-4o 모델 기준 토큰 수 측정

- 스마트 히스토리 압축 : `clean_chat_history()` 1000자 초과 메시지는 앞뒤 500자만 보존하여 핵심 정보 유지

```
[DEBUG] 메시지 토큰 수 조정: 4602 -> 1636
[DEBUG] 라우팅 시작
[DEBUG] 라우팅 요청: 손흥민 정보와 관련 기사 알려줘
[DEBUG] 다중 도구 호출 감지: 2개
```

▶ 에이전트 구현 (추가 기능)

A. 프론트엔드 UI 연동 (예: Streamlit 등)

streamlit을 사용하여 전체 웹 채팅 UI를 구성

B. 다양한 실서비스 사례에 맞는 시나리오

자동화 오케스트레이션 구조를 통한 다중 도구 호출을 사용 → 다양한 시나리오에 대응하도록 구현

C. 오류 및 예외처리

대부분의 주요 함수에 try ... except 구문을 사용

D. 컨텍스트 관리

현재 대화의 맥락을 이해하고 유지하는 세션 기반 컨텍스트 관리 기능

st.session_state.messages를 통해 대화 기록을 세션 내에서 유지

chat_history가 router_agent를 통해 하위 에이전트에 전달 → 이전 대화 내용을 바탕으로 더 정확하고 맥락에 맞는 답변을 생성

▶ 트러블슈팅 (Troubleshooting)

트러블슈팅 사례 ① - 데이터 수집 및 머신러닝

과적합 문제

- 문제 : 초기 머신러닝 모델에서 과적합 발생
- 해결 : `ExtraTreesClassifier`로 모델 변경 → 일반화 성능 개선

전처리 통일 문제

- 문제 : 팀원마다 feature 이름이 다름 → 모델 학습 충돌
- 해결 : 변수 명 규칙을 정의하고 전원 통일 → 협업 시 혼선 최소화

데이터 수집 속도 저하

- 문제 : 경기 통계 수집 시 평균 2분 소요
- 해결 : `Understat` 라이브러리 활용 → 2초 내 수집으로 대폭 개선



Train vs Validation 성능 (mean $\pm$ std)

acc : 0.952 $\pm$ 0.003 | 0.748 $\pm$ 0.029

auc : 0.990 $\pm$ 0.001 | 0.817 $\pm$ 0.024

logloss : -0.224 $\pm$ 0.007 | -0.540 $\pm$ 0.047

트러블슈팅 사례 ② - OpenAI API 응답 처리

타입 인식 오류

- 문제 : `openai.Stream` 응답을 `GeneratorType`으로 검사했으나 if문이 작동하지 않음
- 해결 : 타입 체크 조건을 '`openai.Stream`'로 정확히 지정하여 스트리밍 응답이 정상 처리되도록 수정

▶ 트러블슈팅 (Troubleshooting)

트러블슈팅 사례 ② – OpenAI API 응답 처리

대화 맥락 반영 실패

- 문제 : chat_history를 생성만 하고 실제 라우팅 시 사용하지 않음
- 해결 : 전체 메시지 구성 시 이전 대화 내용 자동 추가, 시스템 메시지 누락 시 자동 삽입, 전체 메시지에 대해 토큰 관리 적용
- 효과 : 다중 턴 질의에서도 자연스럽게 일관된 응답 제공 가능

```
213 def route_query(user_query: str, chat_history: list) -> str:
214     # OpenAI 모델이 도구 사용 여부를 결정하도록 메시지 준비
215
216     messages = [
217         {"role": "system", "content": prompt_templates.ROUTER_SYSTEM_PROMPT},
218
219         {"role": "user", "content": user_query}
220     ]
221
222     # 전체 대화 턴에 대한 토큰 관리
223     messages = token_manager.manage_history_tokens(
224         messages, max_tokens=2000, model_name=model_name
```

```
229 def route_query(user_query: str, chat_history: list) -> str:
230     # OpenAI 모델이 도구 사용 여부를 결정하도록 메시지 준비
231     # chat_history를 기반으로 메시지 구성
232     messages = chat_history.copy() if chat_history else []
233
234     # 시스템 메시지가 없으면 추가
235     if not messages or messages[0].get("role") != "system":
236         messages.insert(0, {"role": "system", "content": prompt_templates.ROUTER_SY
237
238     # 현재 사용자 질문 추가
239     messages.append({"role": "user", "content": user_query})
240
241
242     # 토큰 관리로 적절한 길이로 조정
243     messages = token_manager.manage_history_tokens(
244         messages, max_tokens=2000, model_name=model_name
245
```

▶ 트러블슈팅 (Troubleshooting)

트러블슈팅 사례 ③ – 복합 질의 처리 문제

단일 기능만 처리

- 문제 : “이번 경기 요약과 살라 선수 요약해줘” 같은 복합 질문에도 하나의 기능만 실행됨
- 원인 : 라우터 구조가 단일 구조만 처리하도록 설계됨
- 해결 : 구조 자체를 수정하여 다중 Function Calling 지원 → 여러 에이전트 동시 호출 및 통합 응답 가능

수정 전

```
if tool_calls:
    # 도구 호출이 있는 경우, 첫 번째 도구 호출을 처리
    tool_call = tool_calls[0]
    ...

final_answer
= available_functions[function_name](user_query, chat_history)
```

수정 후

```
# 도구 호출이 감지되면 실행
if tool_calls:
    # 단일 도구 호출인 경우
    if len(tool_calls) == 1:
        tool_call = tool_calls[0]
    ...
else: # 다중 도구 호출인 경우
    tool_results = []
    for i, tool_call in enumerate(tool_calls)

    # 다중 결과를 OpenAI를 통해 통합
    return integrate_multiple_results(user_query, tool_results, chat_history)
```

▶ 트러블슈팅 (Troubleshooting)

트러블슈팅 사례 ④ - 뉴스 수집 및 다국어 처리

기사 수집 방식 전환

- 문제 : 구글 검색 기반으로 한국어 기사 크롤링에는 성공했으나 저작권 이슈 및 구조 변경에 따른 유지보수 부담 발생
- 원인 : 직접 크롤링 방식 포기, 정식 API 기반 수집 방식으로 변경하여 법적 안정성과 유지보수 용이성 확보
- 해결 : 무료 사용이 가능한 NEWS API로 결정 (Bing API는 신용카드 요구로 사용 불가)

외국어 기사 요약 실패

- 문제 : 영어 기사도 함께 요약하려고 했으나, 자연어 질문에서 정확한 키워드 추출에 실패하여 관련 기사 검색 실패
- 시도 : 예) “손흥민 근황 알려줘” → “Son Heung-Min” 번역 → 뉴스 검색 → 기사 요약 → 다시 한국어 번역
- 결과 : 번역된 키워드가 실제 기사에 반영되지 않아 기사 검색 실패 → 요약 불가
- 최종 결정 : 한국어 기사만 대상으로 요약 진행, 한글 질문 → 한글 기사 검색 → 요약

1. 제목 : 토트넘, 방한 앞두고 손흥민 매각하나...“좋은 제안 오면 위약금 37억원 물고서라도 팔 것”

요약 : 토트넘이 잉글랜드 프리미어리그의 스타 손흥민을 계약 만료 1년을 앞두고 매각할 가능성이 제기됐다. 토트넘홋스퍼뉴스는 토트넘이 손흥민의 의무 출전 계약 위반에 따른 위약금을 감수하고라도

링크 : <https://www.khan.co.kr/article/202506241030001>

[기사감정분석결과] : 부정

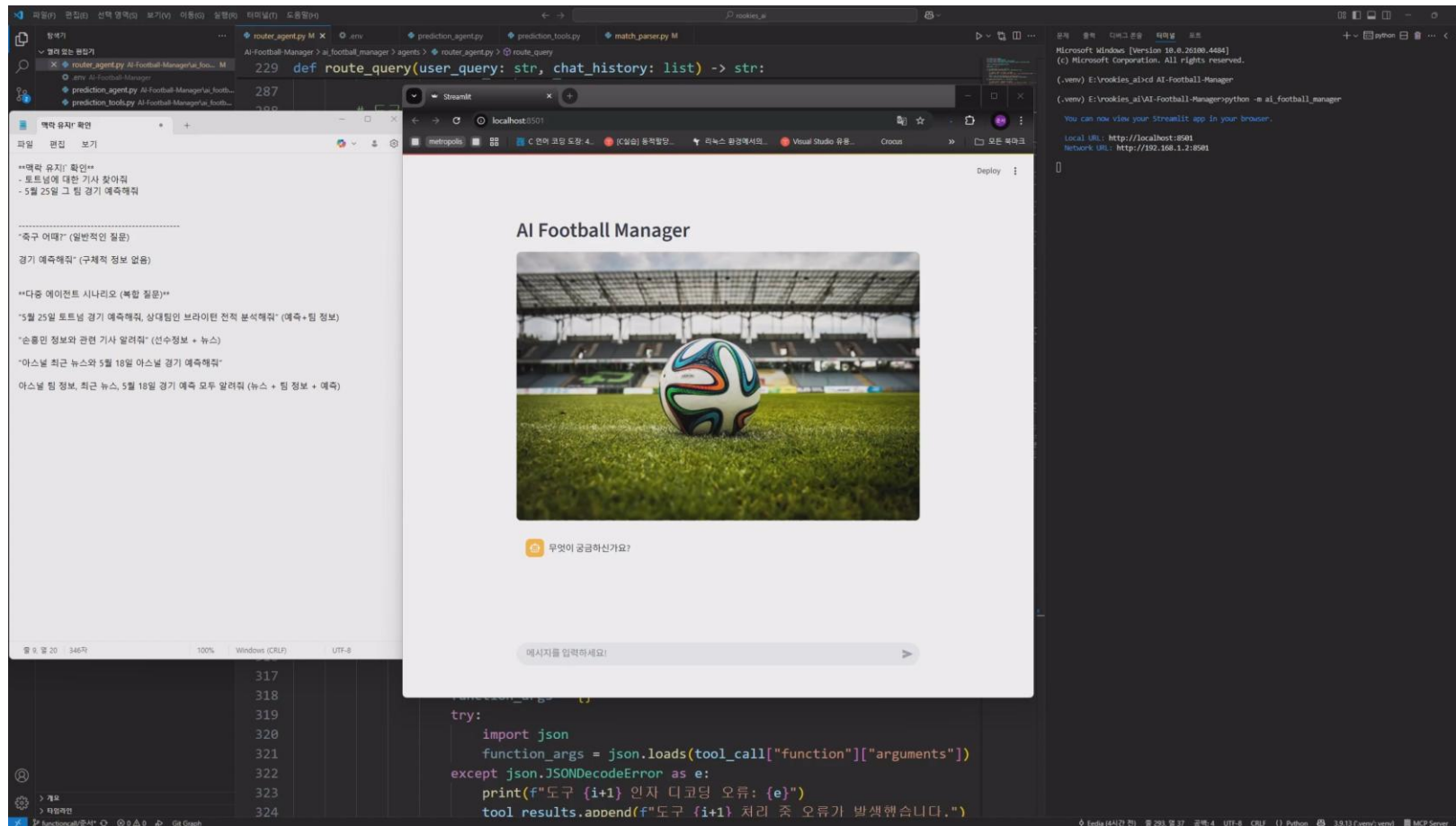
[내가 팬이라면 AI 코멘트] : 손흥민의 이적 가능성 소식은 축구 팬들에게 큰 충격으로 다가올 것 같네요, 그의 미래가 어떻게 될지 궁금합니다.

04

K-Digital Training

프로젝트 수행 경과

▶ 애플리케이션 시연



[영상 링크](#)

프로젝트 결과물에 대한 완성도 평가

- 비록 기획에서 목표로 한 90%의 정확도에 미치지는 못하지만 나름 준수한 정확도(79.7%)를 보임. 또한 디렉토리 구조, 에이전트, 함수가 유기적으로 연결되어 확장이 용이하도록 개발함.
- 원래 계획했던 승부 예측이나 팀/선수 분석에 대한 데이터들을 시각화를 해서 보여주는 기능을 개발하지 못하여 전체 기획에서 90%정도 달성. 완전한 완성도를 달성하지 못함

잘한 부분과 아쉬운 점

- **잘한 점** : 부족한 데이터로 과적합 없이 준수한 정확도 출력, 다중 에이전트 구현으로 한 질문에 여러 요청이 들어와도 처리 가능
- **아쉬운 점** : EPL 예측을 메인으로 하고 있는데 EPL이 한 시즌당 340경기밖에 없어 데이터가 부족, 현재 시즌이 끝난 상태여서 실시간으로 경기 데이터를 확인할 수 없음

프로젝트 결과물의
추후 개선점이나 보완할 점 등 내용 정리

- 야구나 농구 등 다른 스포츠 종목 추가
- 배당률(odds) 관련 데이터 수집하여 학습에 사용
- 뉴스 API에 추가로 다른 API를 사용할 수 있었다면 다양한 뉴스를 탐색할 수 있었을 것

프로젝트를 수행하면서
느낀 점이나 경험한 성과

- **느낀 점** : 팀원 각자 개발한 모듈이 다를 때 이를 총괄하여 통합할 인원이 시작부터 정해져 있어야 충돌 없이 통합이 가능함. 또한 개발 구역이 다르더라도 미리 디렉토리 구조와 변수 명 등을 통일해야 함.

감사합니다



Github